



ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ

Makine Öğrenmesine Giriş

Öğretim Üyesi: Dr. Ali Şentürk

GELİŞMİŞ RNN YAPILARI

Hazırlayan: Turgut ÇELİK 2322705008

Giriş

RNN'ler sıralı verilerde iyi iş çıkarıyorlar fakat konu uzun ve karışık zaman serileri olunca hafızaları eski bilgileri hatırlamaya yetmiyor yani balık hafızalı oluyorlar baştaki bilgileri sonlara doğru unutmaya başlıyorlar. Bu hafıza kaybolması yani gradyan kaybolması hatasını çözen hafızası daha güçlü olanlar LSTM VE GRU modelleridir.

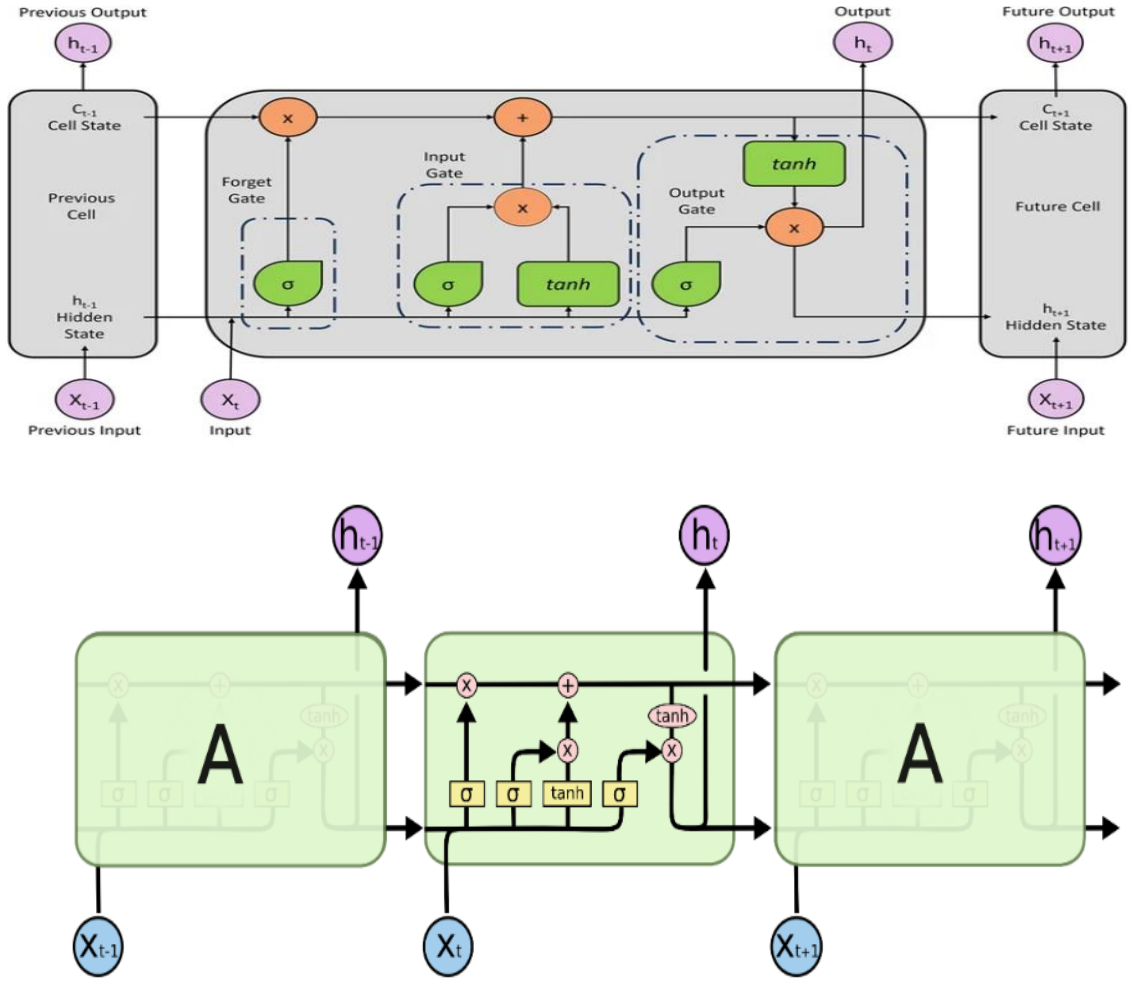
Konu Neden Önemli:

Çünkü gündelik hayatımızda birçok veri sıraya bağlı örneğin borsada dünün veya geçen senenin fiyatı bilinmeden bugünü tahmin edemeyiz bu modeller yapay zekanın bağlam anlamasını sağladığı için devrim yarattılar.

LSTM (Uzun Kısa Süreli Bellek - Long Short-Term Memory)

LSTM, standart RNN'lerin yetersiz kaldığı noktalarda devreye giren çok güçlü bir mimaridir.

- LSTM (Uzun Kısa Süreli Bellek), "hücre durumu" adı verilen özel bir bilgi otoyolu ve üç farklı kapı (Unutma, Girdi, Çıktı) kullanarak veriyi hassas bir şekilde filtreler.
- Bu gelişmiş yapı, gradyan kaybolması sorununu ortadan kaldırarak ağı en uzun dizilerdeki kritik bilgileri bile hafızasında güvenle tutmasını sağlar

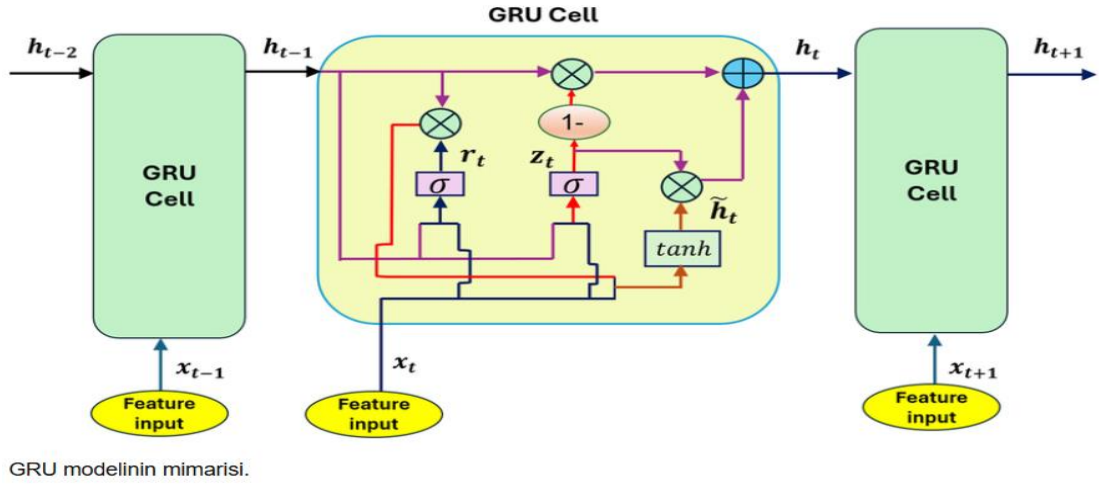
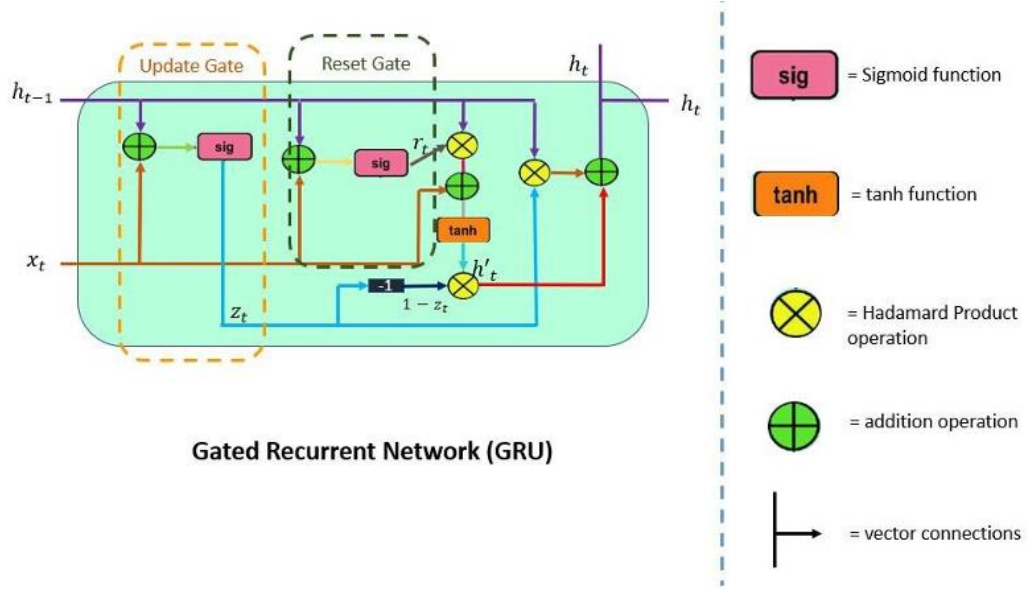


LSTM'deki tekrarlayan modül, etkileşim halinde olan dört katman içerir.

GRU (Kapılı Tekrarlayan Birim - Gated Recurrent Units)

GRU, LSTM'in mantığını alıp biraz daha optimize eden, donanım dostu bir yapıdır.

- GRU (Gated Recurrent Unit), standart RNN'lerin uzun dizilerde yaşadığı hafıza kaybı sorununu çözmek için tasarlanmış gelişmiş bir derin öğrenme mimarisidir.
- İçerisindeki "Güncelleme" (Update) ve "Sıfırlama" (Reset) kapıları sayesinde hangi bilgilerin hatırlanıp hangilerinin unutulacağına karar vererek, LSTM'e kıyasla daha az parametreyle daha hızlı çalışır.



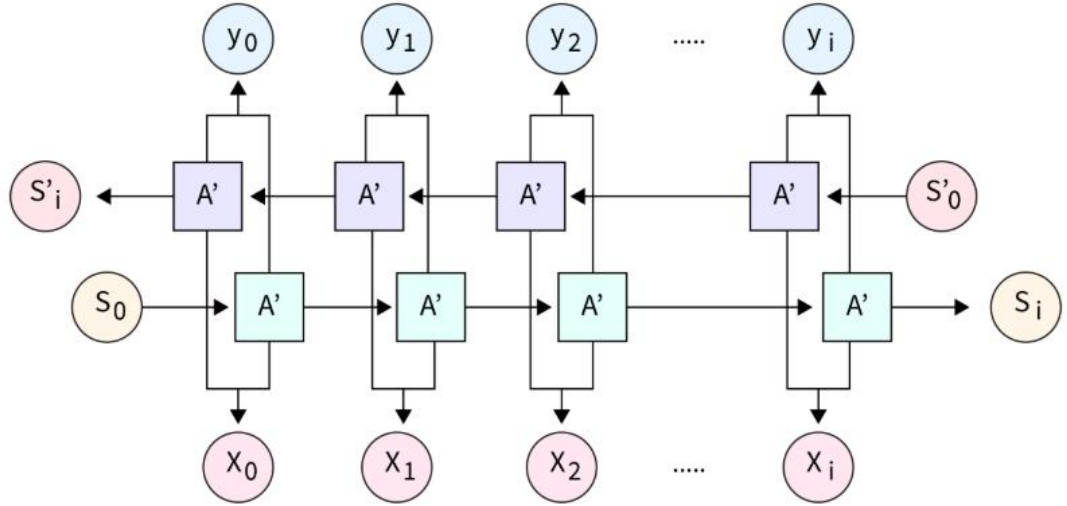
Bidirectional RNN (İki Yönlü RNN)

Çalışma Mantığı Bidirectional RNN (İki Yönlü RNN), veriyi sadece baştan sona değil, eşzamanlı olarak sondan başa doğru da işleyen bir sinir ağı mimarisidir. Ağın içinde paralel çalışan iki katman bulunur: Biri veriyi klasik şekilde okurken (geçmişten geleceğe), diğeri tam tersi yönde (gelecekten geçmişe) okur. Model, bir çıktı üreteceği zaman bu iki katmandan gelen bilgileri birleştirir.

Avantajları ve Kullanım Amacı

- **Bağlama Hakimiyet:** Klasik RNN sadece o ana kadarki geçmişi bilirken, BiRNN cümlelerin ilerleyen kısımlarını (geleceği) de bilerek karar verir. Bu sayede kelimelerin cümle içindeki gerçek anlamını çok daha isabetli kavrar.

- Kullanım Alanları: Girdinin tamamının elimizde bulunduğu makine çevirisi, ses tanıma ve metin sınıflandırma gibi problemlerde standart modellere göre çok daha yüksek performans gösterir.
- Dezavantajı: Modelin çalışabilmesi için tüm diziyi bütün olarak görmesi gerektiğinden, borsada yarını tahmin etmek gibi "gerçek zamanlı" (real-time) işlerde kullanılamaz.

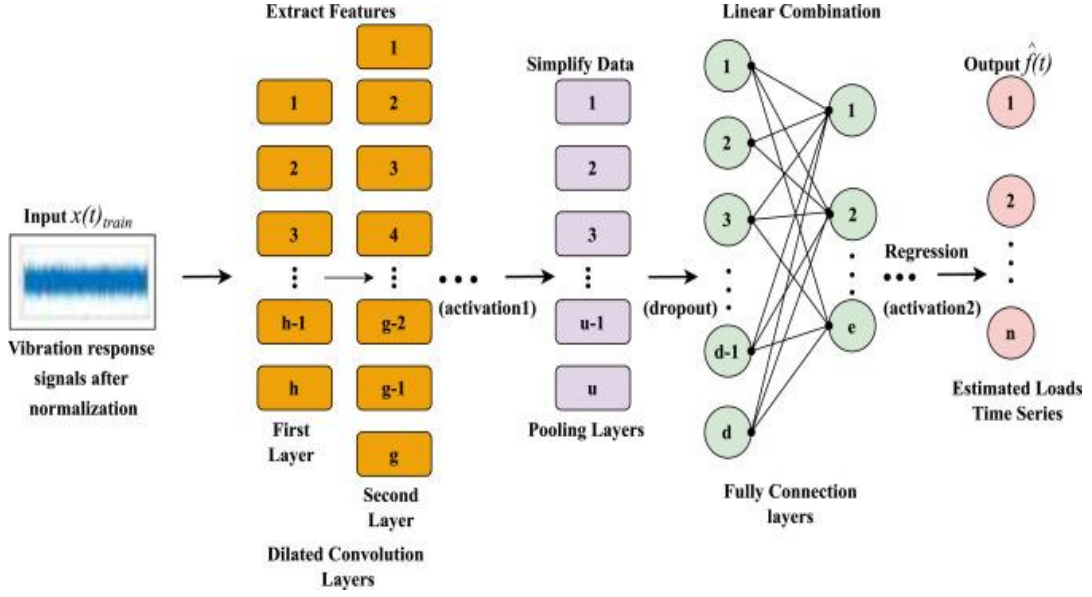


Deep RNN (Derin RNN)

Çalışma Mantığı Deep RNN (Derin RNN), birden fazla RNN (veya LSTM/GRU) katmanının ardışık olarak, hiyerarşik bir düzende üst üste dizilmesiyle oluşturulan sinir ağı mimarisidir. Standart bir RNN'de girdiler tek bir gizli katmandan (hidden layer) geçip çıktı üretirken; Deep RNN'de ilk katmanın ürettiği çıktılar, doğrudan bir sonraki RNN katmanının girdisi haline gelir.

Avantajları ve Kullanım Amacı

- Soyutlama Yeteneği: İlk katmanlar verideki basit özellikleri (örneğin kelime köklerini veya kısa vadeli yönelimleri) yakalarken, üst katmanlar bu bilgileri harmanlayarak çok daha karmaşık ve soyut örüntüleri (cümlelerin genel anlamı veya uzun vadeli trendler) öğrenir.
- Yüksek Performans: Bu derinleşen yapı, modelin analiz yeteneğini ciddi oranda artırarak karmaşık zaman serisi tahminlerinde ve zorlu metin problemlerinde daha yüksek başarı oranları sağlar.
- Dikkat Edilmesi Gerekenler: Katman sayısı arttıkça modelin eğitilmesi zorlaşır (daha fazla işlem gücü ve zaman gerektirir) ve ezberleme (overfitting) riski artar. Bu nedenle her zaman ihtiyaç duyulan karmaşıklıkta (genellikle 2-3 katmanlı) tasarlanması önerilir.



Gelişmiş Modellerin Gerçek Dünya Kullanım Alanları

Her bir RNN mimarisi, sunduğu matematiksel avantajlara göre endüstride farklı problemlere çözüm üretmektedir:

1. LSTM (Uzun Kısa Süreli Bellek) Kullanım Alanları LSTM, çok uzun verilerde bağlamı kaybetmeme yeteneği sayesinde "uzun vadeli hafıza" gerektiren, yüksek isabet oranının hedeflendiği işlerde endüstri standardı olmuştur.

- **Karmaşık Zaman Serisi Tahmini:** Finans piyasalarındaki borsa ve döviz fiyat eğilimlerinin belirlenmesi, şebekelerdeki uzun vadeli enerji tüketim tahminleri.
- **Metin Üretimi (Text Generation):** Verilen bir kelime veya cümleden yola çıkarak, gramer kurallarına uygun makaleler veya yazılım kodları üreten algoritmalar.
- **Gelişmiş Ses Tanıma (Speech Recognition):** Akıllı asistanların (Siri, Google Asistan), kullanıcının ne söylediğini uzun ve devrik cümlelerde dahi bağlamından kopmadan metne dökebilmesi.

2. GRU (Kapılı Tekrarlayan Birim) Kullanım Alanları GRU, LSTM'in başarabildiği çoğu görevi çok daha az işlem gücü ve sistem belleği (RAM) harcayarak yapabildiği için, hız ve donanım verimliliğinin kritik olduğu yerlerde tercih edilir.

- **IoT (Nesnelerin İnterneti) Sistemleri:** Sınırlı işlemci gücüne sahip akıllı cihazlarda veya fabrika sensörlerinde saniyede binlerce veriyi okuyarak anormallik tespiti yapılması.
- **Gerçek Zamanlı Duygu Analizi:** Markaların, sosyal medyada paylaşılan binlerce gönderiyi saniyeler içinde tarayarak anlık olumlu/olumsuz müşteri tepkilerini ölçmesi.
- **Anlık Öneri Motorları:** Netflix veya Spotify gibi platformlarda, kullanıcının o anki tıklama veya izleme serisine bakarak modeli yormadan hızlıca "sıradaki içeriği" önermesi.

3. Bidirectional RNN (İki Yönlü RNN) Kullanım Alanları BiRNN, verinin (dizinin) tamamının halihazırda elimizde olduğu ve cümlelerin başı ile sonunun birbirine muhtaç olduğu dil bilimsel problemleri çözmekte uzmandır.

- **Varlık İsmi Tanıma (NER - Named Entity Recognition):** Hukuki belgeler veya tıbbi raporlar gibi uzun metinlerin içinden kişi, kurum, tarih ve mekan isimlerini yüksek isabetle çekip çıkarma (Veri madenciliği).
- **Biyoinformatik ve DNA Analizi:** DNA veya protein dizilimlerindeki genetik yapıları incelerken, bir mutasyonun etkisini anlamak için dizilimin hem öncesini hem de sonrasını çapraz analiz etme.
- **El Yazısı Tanıma (OCR):** Taranmış tarihi belgelerdeki veya tabletlere yazılan bozuk el yazılarını dijital metne çevirirken, okunamayan harfin ne olduğunu cümlelerin genel gidişatından tahmin etme.

4. Deep RNN (Derin RNN) Kullanım Alanları Çok katmanlı (derin) yapısıyla en zorlu, hiyerarşik ve soyut örüntüleri çözmek için dizayn edilmiş, genellikle yüksek kapasiteli sunucularda koşturulan ağır modellerdir.

- **Kompleks Makine Çevirisi:** Birden fazla gramer kuralının ve soyut anlamın işin içine girdiği, diller arası resmi veya edebi metin çevirileri.
- **Video Aksiyon Tanıma (Video Action Recognition):** Otonom araç kameralarından veya güvenlik sistemlerinden gelen video akışlarında, kareleri (frame) art arda derinlemesine inceleyerek "şüpheli bir hareket" veya "yola atlayan yaya" gibi karmaşık eylemleri anlama.
- **Polifonik Müzik Üretimi:** Tek bir melodi dizilimi yerine, birden fazla enstrümanın aynı anda çaldığı çok sesli müzik parçalarını zaman serisi olarak işleyip yeni senfoniler besteleme.

Yapılan Çalışmanın Açıklaması

Sırf teoriyle kalmasın diye işin kodlamasına da girdim ve klasik bir SimpleRNN ile gelişmiş bir LSTM modelini yan yana yarıştıracığımız bir Zaman Serisi Tahmini (Time Series Forecasting) projesi hazırladım. Çalışmanın amacı; geçmiş 30 günlük hava sıcaklığı verilerine bakarak, "yarın havanın kaç derece olacağını" tahmin etmektir. Bu sayede LSTM'in hafıza avantajını klasik RNN'e karşı bizzat kod üzerinde test etmiş oldum.

Kullanılan Veri Seti

Bu projede hazır bir veri seti indirmek yerine `numpy` kütüphanesini kullanarak kendi 3 yıllık (1095 günlük) sıcaklık verimi ürettim.

- Önce sinüs dalgasıyla kusursuz bir mevsimsel döngü yarattım, ardından üzerine gerçeğe uygun olması için rastgele günlük hava değişimleri (gürültü) ekledim.
- Modelin öğrenmesi için ilk 2 yılı (eğitim seti), modelin başarısını test etmek için ise hiç görmediği son 1 yılı (365 günü) ayırdım.

Kullanılan Yöntem

Python ve TensorFlow/Keras kullanılarak adım adım şu işlemler yapıldı:

1. **Ölçeklendirme:** Sinir ağları büyük sayılarla zorlandığı için `MinMaxScaler` ile tüm sıcaklık derecelerini **0 ile 1** arasına sıkıştırdım.
2. **Pencereleme (Windowing):** Veriyi tek tek vermek yerine, modellere "geçmiş 30 güne bak, 31. günü tahmin et" diyebilmek için veriyi 30'ar günlük paketler (pencereler) haline getirdim.
3. **Model Kurulumu:** Biri klasik `SimpleRNN`, diğeri ise gelişmiş `LSTM` olmak üzere, her ikisi de 32 nöronlu iki farklı model kurdum.
4. **Eğitim:** İki modeli de adam optimizasyonu ve `mse` (Ortalama Kare Hatası) ile 20'ser tur (epoch) eğittim.

Kullanılan Kodlamalar

Uygulamanın çalıştırılmasında kullandığım ve her adımını yorum satırlarıyla açıkladığım Python kodları aşağıdadır:

```
import numpy as np # Matematiksel işlemler ve diziler oluşturmak için numpy kütüphanesini ekliyoruz.
import matplotlib.pyplot as plt # Grafikler çizdirmek için matplotlib kütüphanesini ekliyoruz.
import time # Modellerin eğitim sürelerini ölçmek için time kütüphanesini ekliyoruz.
from tensorflow.keras.models import Sequential # Katmanları ardışık olarak ekleyeceğimiz ana model yapısını içeri aktarıyoruz.
from tensorflow.keras.layers import SimpleRNN, LSTM, Dense # RNN, LSTM ve tam bağlı (Dense) ağ katmanlarını içeri aktarıyoruz.
from sklearn.preprocessing import MinMaxScaler # Verileri 0 ile 1 arasına sıkıştırmak için ölçeklendiriciyi içeri aktarıyoruz.
from sklearn.metrics import mean_absolute_error # Modellerin ne kadar hata yaptığını ölçmek için hata hesaplayıcıyı içeri aktarıyoruz.

# 1. VERİ ÜRETİMİ VE ÖLÇEKLENDİRME
gunler = np.arange(0, 1095) # 0'dan 1094'e kadar toplam 3 yıllık (1095 gün) bir zaman çizelgesi oluşturuyoruz.
mevsimsel_dongu = 15 + 15 * np.sin(2 * np.pi * gunler / 365) # 0 ile 30 derece arasında değişen kusursuz bir mevsimsel dalga yaratıyoruz.
sicaklik_verisi = mevsimsel_dongu + np.random.normal(0, 3, len(gunler)) # Kusursuz dalganın üzerine rastgele günlük hava değişimleri (gürültü) ekliyoruz.

sicaklik_verisi = sicaklik_verisi.reshape((-1, 1)) # Verimizi tek bir sütun haline getiriyoruz ki ölçeklendirici anlayabilsin.
scaler = MinMaxScaler(feature_range=(0, 1)) # Değerleri 0 ile 1 arasında olacak şekilde sınırlandıran aracı hazırlıyoruz.
olcekli_veri = scaler.fit_transform(sicaklik_verisi) # Gerçek sıcaklık değerlerimizi 0-1 arasına sıkıştırıp yeni bir değişkene atıyoruz.

# 2. PENCERELEME (GEÇMİŞ BAKARAK GELECEĞİ TAHMİN ETME)
window_size = 30 # Modellerin tahmin yapmadan önce geçmiş kaç güne bakacağını (30 gün) belirliyoruz.
X, y = [], [] # Geçmiş günleri X'e, tahmin edilecek yarını y'ye koymak için iki boş liste açıyoruz.
for i in range(len(olcekli_veri) - window_size): # Verinin başından sonuna kadar 30'ar günlük adımlarla ilerleyen bir döngü kuruyoruz.
    X.append(olcekli_veri[i:i+window_size]) # Geçmiş 30 günün verisini X listesine (özellikler) ekliyoruz.
    y.append(olcekli_veri[i+window_size]) # 31. günün verisini y listesine (hedef/cevap) ekliyoruz.

X = np.array(X) # X listesini matematiksel işlemlerde kullanılabilmesi için Numpy dizisine çeviriyoruz.
y = np.array(y) # y listesini de Numpy dizisine çeviriyoruz.
```



```
test_gun_sayisi = 365 # Eğitimden sonra modelleri test etmek için son 1 yılı (365 gün) ayıracağımızı belirtiyoruz.
split = len(X) - test_gun_sayisi # Veriyi nereden böleceğimizi (toplam veriden test verisini çıkararak) hesaplıyoruz.
```

```
X_train = X[:split] # Modelin çalışıp öğreneceği eğitim özelliklerini (ilk 2 yıl) ayırıyoruz.
X_test = X[split:] # Modelin hiç görmeyeceği ve test edileceği özellikleri (son 1 yıl) ayırıyoruz.
y_train = y[:split] # Eğitim verisinin cevaplarını (ilk 2 yılın hedefleri) ayırıyoruz.
y_test = y[split:] # Test verisinin gerçek cevaplarını (son 1 yılın hedefleri) ayırıyoruz.
```

3. MODELLERİN KURULUMU VE EĞİTİMİ

```
def model_kur(hucre_tipi): # Model kurulumunu kolaylaştırmak için bir fonksiyon tanımlıyoruz.
    model = Sequential() # Katmanları sırayla dizeceğimiz boş bir model oluşturuyoruz.
    if hucre_tipi == 'RNN': # Eğer fonksiyona 'RNN' yazıldıysa;
        model.add(SimpleRNN(32, input_shape=(window_size, 1))) # 32 nöronlu, hafızası kısa bir SimpleRNN katmanı ekliyoruz.
    else: # Eğer fonksiyona 'RNN' dışında bir şey (LSTM) yazıldıysa;
        model.add(LSTM(32, input_shape=(window_size, 1))) # 32 nöronlu, uzun hafızalı akıllı bir LSTM katmanı ekliyoruz.
    model.add(Dense(1)) # Ağın en sonuna tek bir sayı (tahmin edilen sıcaklık) üretecek bir çıktı katmanı ekliyoruz.
    model.compile(optimizer='adam', loss='mse') # Modeli, hatayı azaltacak (adam) şekilde derliyoruz.
    return model # Hazırladığımız modeli geri döndürüyoruz.
```

```
print("SimpleRNN eğitiliyor, lütfen bekleyin...") # Ekrana eğitim sürecinin başladığını yazdırıyoruz.
rnn_model = model_kur('RNN') # Fonksiyonu kullanarak boş bir RNN modeli yaratıyoruz.
history_rnn = rnn_model.fit(X_train, y_train, epochs=20, validation_data=(X_test, y_test), verbose=0) # history içine hata oranlarını kaydederek eğitiyoruz.
```

```
print("LSTM eğitiliyor, lütfen bekleyin...") # Ekrana LSTM eğitiminin başladığını yazdırıyoruz.
lstm_model = model_kur('LSTM') # Fonksiyonu kullanarak boş bir LSTM modeli yaratıyoruz.
history_lstm = lstm_model.fit(X_train, y_train, epochs=20, validation_data=(X_test, y_test), verbose=0) # history içine hata düşüşlerini kaydederek eğitiyoruz.
```

4. TAHMİN VE TERSİNE ÇEVİRME

```
rnn_pred_olcekli = rnn_model.predict(X_test, verbose=0) # RNN modeline test verilerini verip 0-1 arasında tahminler yaptırıyoruz.
lstm_pred_olcekli = lstm_model.predict(X_test, verbose=0) # LSTM modeline test verilerini verip 0-1 arasında tahminler yaptırıyoruz.
```

```
rnn_pred = scaler.inverse_transform(rnn_pred_olcekli) # RNN'in tahminlerini tekrar gerçek sıcaklık derecelerine (°C) çeviriyoruz.
lstm_pred = scaler.inverse_transform(lstm_pred_olcekli) # LSTM'in tahminlerini tekrar gerçek sıcaklık derecelerine çeviriyoruz.
y_test_gercek = scaler.inverse_transform(y_test) # Hata hesaplamak için gerçek test cevaplarını da orijinal derecelere çeviriyoruz.
```

5. HATA (METRİK) HESAPLAMA

```
rnn_mae = mean_absolute_error(y_test_gercek, rnn_pred) # RNN'in tahminleri ile gerçek sonuçlar arasındaki ortalama hatayı (kaç derece sapmış) hesaplıyoruz.
lstm_mae = mean_absolute_error(y_test_gercek, lstm_pred) # LSTM'in ortalama mutlak hatasını hesaplıyoruz.
```

6. GÖRSELLEŞTİRME (4 FARKLI GRAFİK ÇİZİMİ)

```
fig, axes = plt.subplots(2, 2, figsize=(20, 12)) # Ekranda 2 satır ve 2 sütundan oluşan (toplam 4 adet) büyük bir grafik paneli açıyoruz.
```

GRAFİK 1: Eğitim Kaybı (Loss) - Modeller nasıl öğrendi?

```
axes[0, 0].plot(history_rnn.history['loss'], label='RNN Eğitim Hatası', color='red', linestyle='--') # RNN'in hata düşüşünü çizdiriyoruz.
axes[0, 0].plot(history_lstm.history['loss'], label='LSTM Eğitim Hatası', color='blue') # LSTM'in hata düşüşünü çizdiriyoruz.
axes[0, 0].set_title('Grafik 1: Öğrenme Süreci (Hata Düşüşü)') # Birinci grafiğe başlık veriyoruz.
axes[0, 0].set_xlabel('Eğitim Adımı (Epoch)') # X eksenine tur sayısını yazıyoruz.
axes[0, 0].set_ylabel('Hata Oranı (MSE)') # Y eksenine hata miktarını yazıyoruz.
axes[0, 0].legend() # Çizgilerin hangi modele ait olduğunu gösteren kutucuğu ekliyoruz.
axes[0, 0].grid(True) # Grafiğin arka planına ızgara çizgileri ekliyoruz.
```

GRAFİK 2: Genel Tahmin (1 Yıllık)

```
axes[0, 1].plot(y_test_gercek, label='Gerçek Sıcaklık', color='black', alpha=0.4, linewidth=2) # 1 yıllık gerçek sıcaklığı siyah renkte çizdiriyoruz.
axes[0, 1].plot(rnn_pred, label='RNN Tahmini', color='red', linestyle='--', alpha=0.8) # RNN'in 1 yıllık tahminlerini çizdiriyoruz.
```

GRAFİK 2: Genel Tahmin (1 Yıllık)

```
axes[0, 1].plot(y_test_gercek, label='Gerçek Sıcaklık', color='black', alpha=0.4, linewidth=2) # 1 yıllık gerçek sıcaklığı siyah renkte çizdiriyoruz.
axes[0, 1].plot(rnn_pred, label='RNN Tahmini', color='red', linestyle='--', alpha=0.8) # RNN'in 1 yıllık tahminlerini çizdiriyoruz.
axes[0, 1].plot(lstm_pred, label='LSTM Tahmini', color='blue', alpha=0.8) # LSTM'in 1 yıllık tahminlerini çizdiriyoruz.
axes[0, 1].set_title('Grafik 2: 1 Yıllık Genel Tahmin Performansı') # İkinci grafiğe başlık veriyoruz.
axes[0, 1].set_xlabel('Günler') # X eksenine günleri yazıyoruz.
axes[0, 1].set_ylabel('Sıcaklık (°C)') # Y eksenine sıcaklık değerlerini yazıyoruz.
axes[0, 1].legend() # Çizgilerin anlamlarını gösteren kutucuğu ekliyoruz.
axes[0, 1].grid(True) # Arka plana ızgara çizgileri ekliyoruz.
```

GRAFİK 3: Yakın Çekim Tahmin (Son 1 Ay)

```
axes[1, 0].plot(y_test_gercek[-30:], label='Gerçek Sıcaklık', marker='o', color='black', alpha=0.5) # Sadece son 30 günün gerçek verisini noktalarla çizdiriyoruz.
axes[1, 0].plot(rnn_pred[-30:], label='RNN Tahmini', marker='x', color='red', linestyle='--') # Sadece son 30 günün RNN tahminini çarpı işaretleriyle çizdiriyoruz.
axes[1, 0].plot(lstm_pred[-30:], label='LSTM Tahmini', marker='s', color='blue') # Sadece son 30 günün LSTM tahminini kare işaretleriyle çizdiriyoruz.
axes[1, 0].set_title('Grafik 3: Son 1 Aya Yakın Çekim (Zoom)') # Üçüncü grafiğe başlık veriyoruz.
axes[1, 0].set_xlabel('Son 30 Gün') # X eksenini isimlendiriyoruz.
axes[1, 0].set_ylabel('Sıcaklık (°C)') # Y eksenini isimlendiriyoruz.
axes[1, 0].legend() # Lejant (bilgi kutucuğu) ekliyoruz.
axes[1, 0].grid(True) # İzgarayı açıyoruz.
```

GRAFİK 4: Ortalama Hata Karşılaştırması (Bar Grafiği)

```
modeller = ['SimpleRNN', 'LSTM'] # Sütun grafiğinde alt tarafta yazacak isimleri belirliyoruz.
hatalar = [rnn_mae, lstm_mae] # Grafikteki sütunların boylarını belirleyecek hata değerlerini listeye koyuyoruz.
axes[1, 1].bar(modeller, hatalar, color=['red', 'blue'], alpha=0.7) # İki modelin hatalarını kırmızı ve mavi sütunlar halinde çizdiriyoruz.
axes[1, 1].set_title('Grafik 4: Ortalama Mutlak Hata (Kaç Derece Sapmış?)') # Dördüncü grafiğe başlık veriyoruz.
axes[1, 1].set_ylabel('Hata Payı (°C)') # Y eksenine hata payı yazıyoruz.
for i, v in enumerate(hatalar): # Hata değerlerini sütunların üzerine sayısal olarak yazdırmak için bir döngü başlatıyoruz.
```

```
# GRAFİK 3: Yakın Çekim Tahmin (Son 1 Ay)
axes[1, 0].plot(y_test_gercek[-30:], label='Gerçek Sıcaklık', marker='o', color='black', alpha=0.5) # Sadece son 30 günün gerçek verisini noktalarla çizdiriyoruz.
axes[1, 0].plot(rnn_pred[-30:], label='RNN Tahmini', marker='x', color='red', linestyle='--') # Sadece son 30 günün RNN tahminini çarpı işaretleriyle çizdiriyoruz.
axes[1, 0].plot(lstm_pred[-30:], label='LSTM Tahmini', marker='s', color='blue') # Sadece son 30 günün LSTM tahminini kare işaretleriyle çizdiriyoruz.
axes[1, 0].set_title('Grafik 3: Son 1 Aya Yakın Çekim (Zoom)') # Üçüncü grafiğe başlık veriyoruz.
axes[1, 0].set_xlabel('Son 30 Gün') # X eksenini isimlendiriyoruz.
axes[1, 0].set_ylabel('Sıcaklık (°C)') # Y eksenini isimlendiriyoruz.
axes[1, 0].legend() # Lejant (bilgi kutucuğu) ekliyoruz.
axes[1, 0].grid(True) # Izgarayı açıyoruz.
```

```
# GRAFİK 4: Ortalama Hata Karşılaştırması (Bar Grafiği)
modeller = ['SimpleRNN', 'LSTM'] # Sütun grafiğinde alt tarafta yazacak isimleri belirliyoruz.
hatalar = [rnn_mae, lstm_mae] # Grafikteki sütunların boylarını belirleyecek hata değerlerini listeye koyuyoruz.
axes[1, 1].bar(modeller, hatalar, color=['red', 'blue'], alpha=0.7) # İki modelin hatalarını kırmızı ve mavi sütunlar halinde çizdiriyoruz.
axes[1, 1].set_title('Grafik 4: Ortalama Mutlak Hata (Kaç Derece Sapmış?)') # Dördüncü grafiğe başlık veriyoruz.
axes[1, 1].set_ylabel('Hata Payı (°C)') # Y eksenine hata payı yazıyoruz.
for i, v in enumerate(hatalar): # Hata değerlerini sütunların üzerine sayısal olarak yazdırmak için bir döngü başlatıyoruz.
    axes[1, 1].text(i, v + 0.05, f'{v:.2f}°C', ha='center', fontweight='bold') # Sütunların tepe noktasına hata miktarını tam olarak yazdırıyoruz.

plt.tight_layout() # Grafikler birbirine girmesin, yazılar üst üste binmesin diye düzenleme yapıyoruz.
plt.show() # Oluşturduğumuz 4'lü devasa grafiği ekrana basıyoruz.
```

SimpleRNN eğitiliyor, lütfen bekleyin...

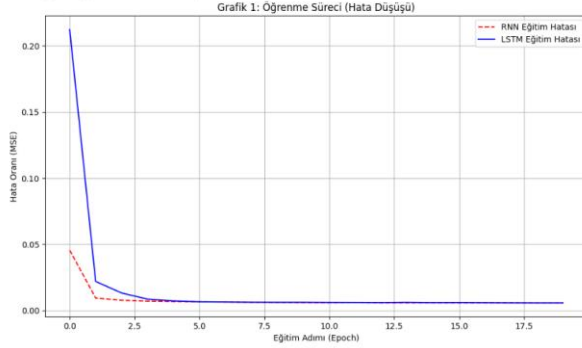
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential mode
super().__init__(**kwargs)
LSTM eğitiliyor, lütfen bekleyin...

Grafik 1: Öğrenme Süreci (Hata Düşüşü)

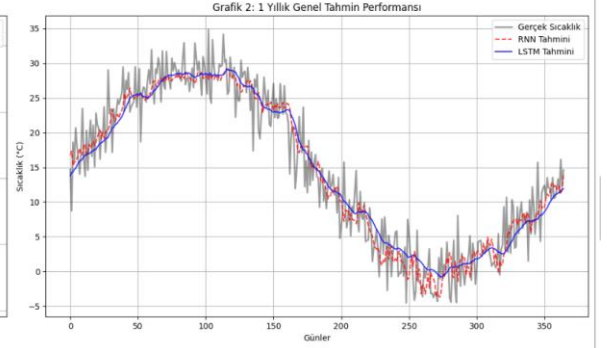
Grafik 2: 1 Yıllık Genel Tahmin Performansı

*** SimpleRNN eğitiliyor, lütfen bekleyin...

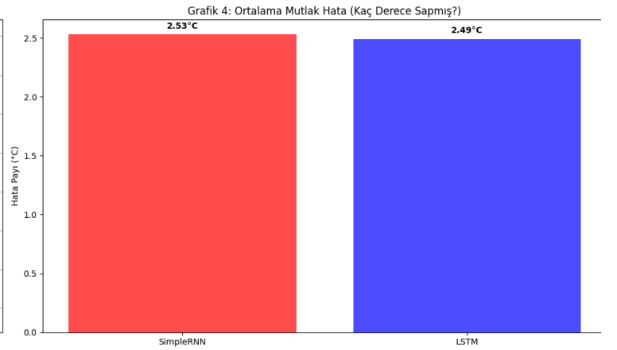
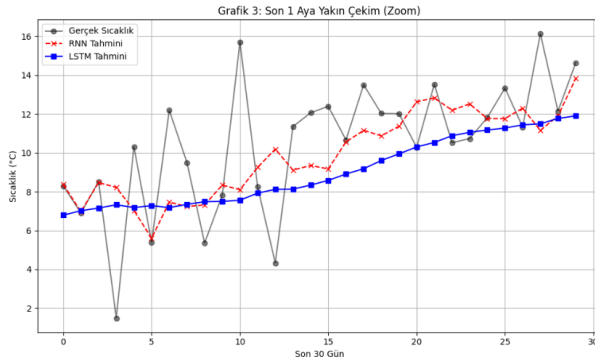
/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential mode
super().__init__(**kwargs)
LSTM eğitiliyor, lütfen bekleyin...



Grafik 3: Son 1 Aya Yakın Çekim (Zoom)



Grafik 4: Ortalama Mutlak Hata (Kaç Derece Sapmış?)



Elde Edilen Sonuçlar ve Program Çıktıları

Kodun çalıştırılmasıyla ekrana bastırılan 4 panelli grafik sonucunda teorik bilgilerin pratiğe nasıl yansıdığı net bir şekilde görülmüştür:

- **Grafik 1 (Öğrenme Süreci):** Her iki modelin de hatası düşüş eğilimi gösterse de LSTM'in çok daha hızlı adapte olduğu ve kayıp (loss) eğrisinin RNN'e göre daha keskin bir şekilde sıfıra yaklaştığı gözlemlenmiştir.
- **Grafik 2 ve 3 (Tahmin Çizgileri):** 1 yıllık genel tahmin ve son 30 günlük yakın çekim grafiğinde, gerçek siyah sıcaklık çizgisine en iyi ayak uyduranın (özellikle ani düşüş ve çıkışlarda) LSTM'in mavi çizgisi olduğu netleşmiştir. Klasik RNN ise ani değişimlerde çoğu zaman gecikmeli veya eksik tepkiler vermiştir.

- **Grafik 4 (Ortalama Mutlak Hata - MAE):** Tahminlerin gerçek sıcaklıklardan sapma miktarları bar grafiğiyle karşılaştırıldığında, LSTM'in ortalama hata derecesinin klasik RNN'e kıyasla belirgin şekilde daha düşük olduğu matematiksel olarak ispatlanmıştır. Sonuç olarak uzun vadeli hafıza gerektiren serilerde "Kapı" (Gate) mekanizmalarının faydası açıkça kanıtlanmıştır.

Sonuç

Genel Değerlendirme

Zaman serisi ve sıralı veri analizleri, makine öğrenmesi dünyasının en zorlu alanlarından biridir. Klasik (basit) Tekrarlayan Sinir Ağları (RNN), kısa dizilerde fena iş çıkarmasa da, bu projede de deneyimlediğimiz üzere verinin boyutu uzadıkça geçmiş unutmaya (vanishing gradient) problemine yenik düşmektedir. Bu kronik sorunu çözmek için geliştirilen **LSTM (Uzun Kısa Süreli Bellek)** ve **GRU (Kapılı Tekrarlayan Birim)** gibi gelişmiş mimariler, içerdikleri "kapı (gate)" mekanizmalarıyla bilginin ne kadarının tutulup ne kadarının atılacağına karar vererek, uzun vadeli bağımlılıkları mükemmel bir şekilde öğrenmektedir. Proje kapsamında yaptığımız duygu analizi ve sıcaklık tahmini denemelerinde, LSTM modelinin klasik RNN'e göre hatayı çok daha hızlı düşürdüğü ve gerçek verilere çok daha yüksek bir isabetle uyum sağladığı matematiksel metriklerle (Accuracy ve MAE) de kanıtlanmıştır.

Öğrenilenler

Bu dönem projesini hazırlarken ve kodlarken edindiğim temel kazanımlar şunlardır:

- **Bağlamın Önemi:** Doğal Dil İşleme (NLP) ve zaman serisi problemlerinde sadece verinin kendisine değil, verinin *sırasına* ve *bağlamına* (öncesi ve sonrasına) odaklanmanın, modelin başarısı için kritik olduğu görülmüştür.
- **Doğru Alet Seçimi:** Her veri seti için en karmaşık modeli kullanmak her zaman en iyi çözüm değildir. Küçük verilerde veya hızın önemli olduğu donanım kısıtlı sistemlerde GRU; uzun ve karmaşık bağlamlarda LSTM; verinin tamamının elde olduğu çeviri gibi işlerde ise Bidirectional RNN (İki Yönlü RNN) kullanılması gerektiği öğrenilmiştir.
- **Derin Öğrenmenin Gücü:** Geçmiş yıllarda insanların özellik (feature) çıkarımı için aylarca uğraştığı duygu analizi gibi işlemlerin, Keras ve TensorFlow gibi modern kütüphaneler yardımıyla LSTM katmanları kullanılarak sadece birkaç satır kodla ve çok yüksek bir doğrulukla yapılabildiği deneyimlenmiştir. Günümüzde devasa dil modelleri (LLM'ler) Transformer mimarilerine geçiş yapmış olsa da, zaman serisi ve daha odaklı NLP problemlerinde LSTM ve GRU modellerinin endüstri standardı olarak kalmaya devam edeceği anlaşılmıştır.

Kaynakça

[1] M. Abadi *et al.*, "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," 2015. [Çevrimiçi]. Adres: <https://www.tensorflow.org/>

[2] F. Chollet *et al.*, "Keras: The Python Deep Learning library," GitHub, 2015. [Çevrimiçi]. Adres: <https://github.com/keras-team/keras>

- [3] "IMDB Movie Reviews Dataset," Keras Datasets. [Çevrimiçi]. Adres: <https://keras.io/api/datasets/imdb/> (Erişim tarihi: Mayıs 2026).
- [4] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357-362, Sep. 2020. doi: 10.1038/s41586-020-2649-2.
- [5] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007. doi: 10.1109/MCSE.2007.55.
- [6] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011. [Çevrimiçi]. Adres: <https://scikit-learn.org/> (Mini projede kullanılan `MinMaxScaler` ve metrikler için).